

NAME : P SAI VENKAT

ROLL NO : 2403A510G0

ASSIGNMENT : 14.3

Task Description #1 – AI-generated HTML Page

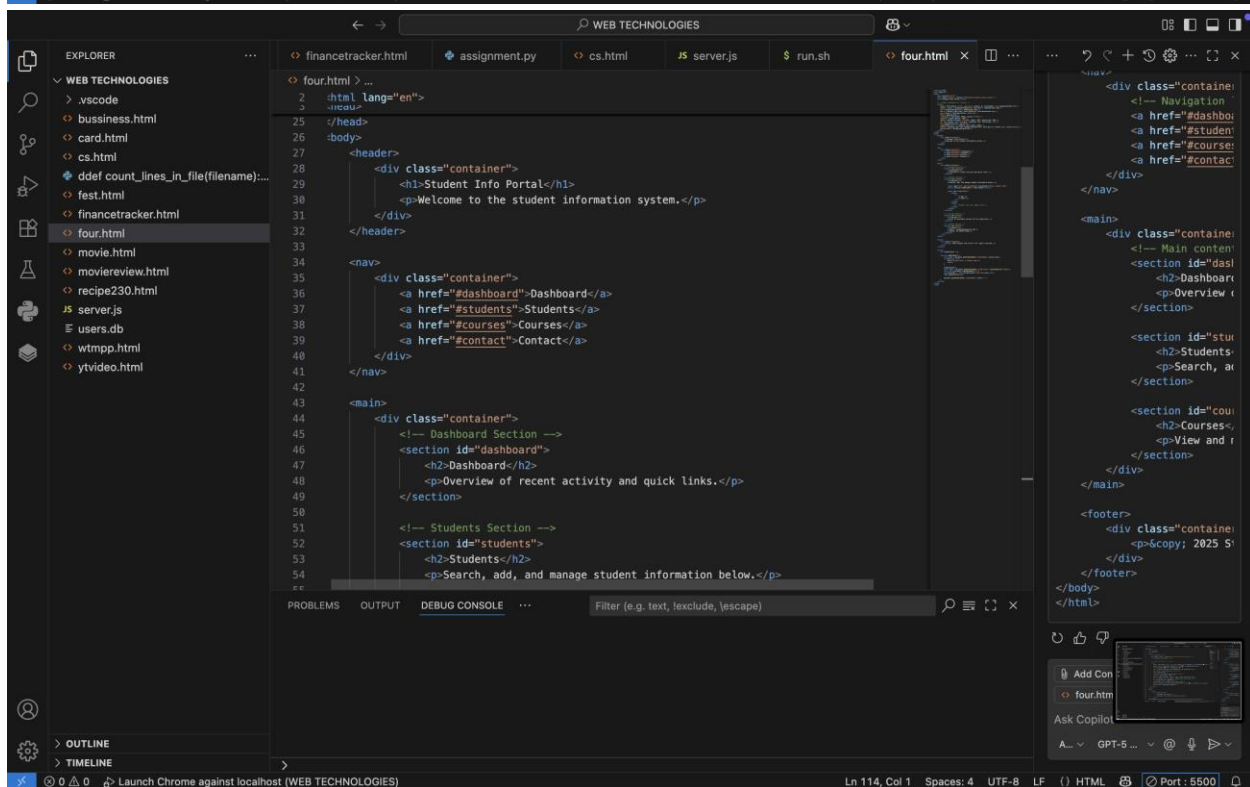
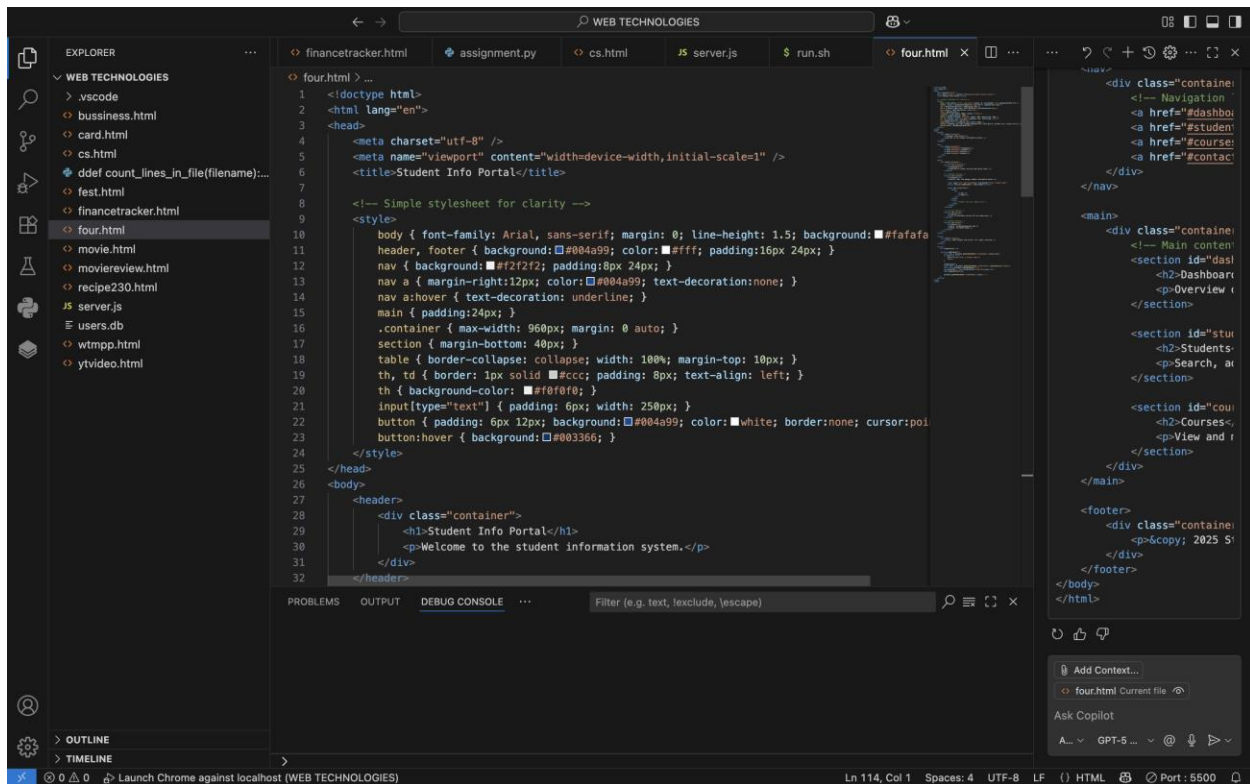
Task: Ask AI to generate a simple HTML homepage for a "Student Info Portal" with a header, navigation menu, and footer.

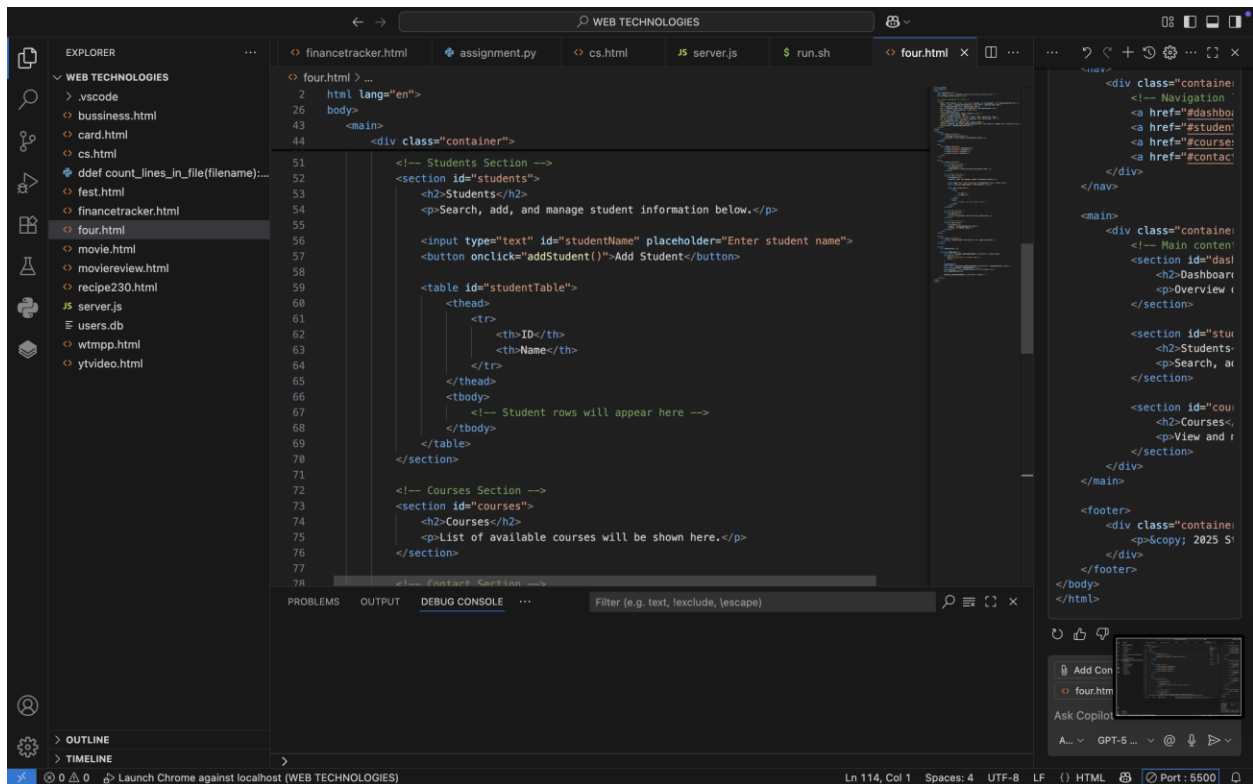
Expected Output:

- HTML code with <header>, <nav>, <footer>.
- Clean indentation, proper tags, and comments.

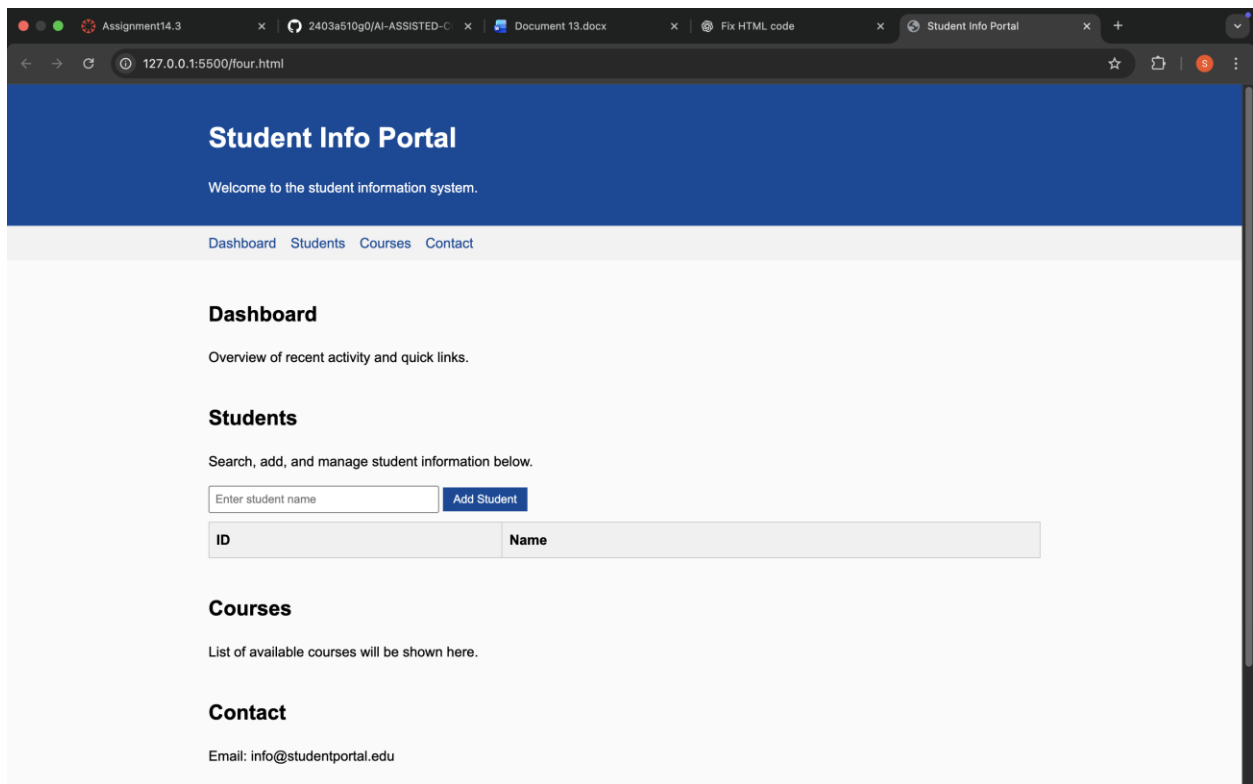
PROMPT USED : To generate a HTML page with a header,navigation menu, and footer for student info portal

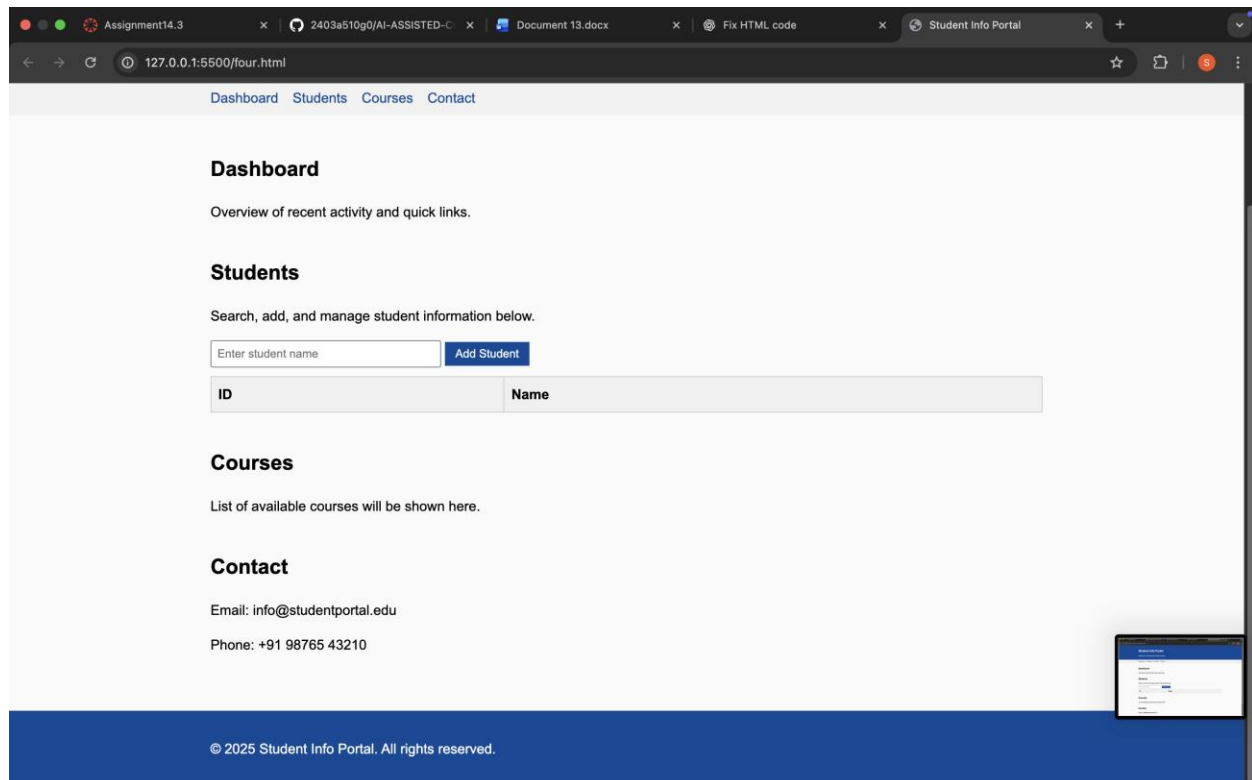
CODE GENERATED :





OUTPUT :





OBSERVATION :

Student Info Portal (simple version)

Structure: header, nav, main, footer present and semantically correct.

Styling: inline `<style>` with basic colors and layout; minimal responsive handling.

Navigation: static links (no JS); good for simple pages and readability.

Accessibility: basic — no aria attributes or a skip link.

Behavior: no dynamic features; ideal as a lightweight starter template.

Task Description #2 – CSS Styling

Task:

Use AI to add CSS styling to Task #1 homepage for:

- Responsive navigation bar.
- Centered content section.

- Footer with light gray background.

Expected Output:

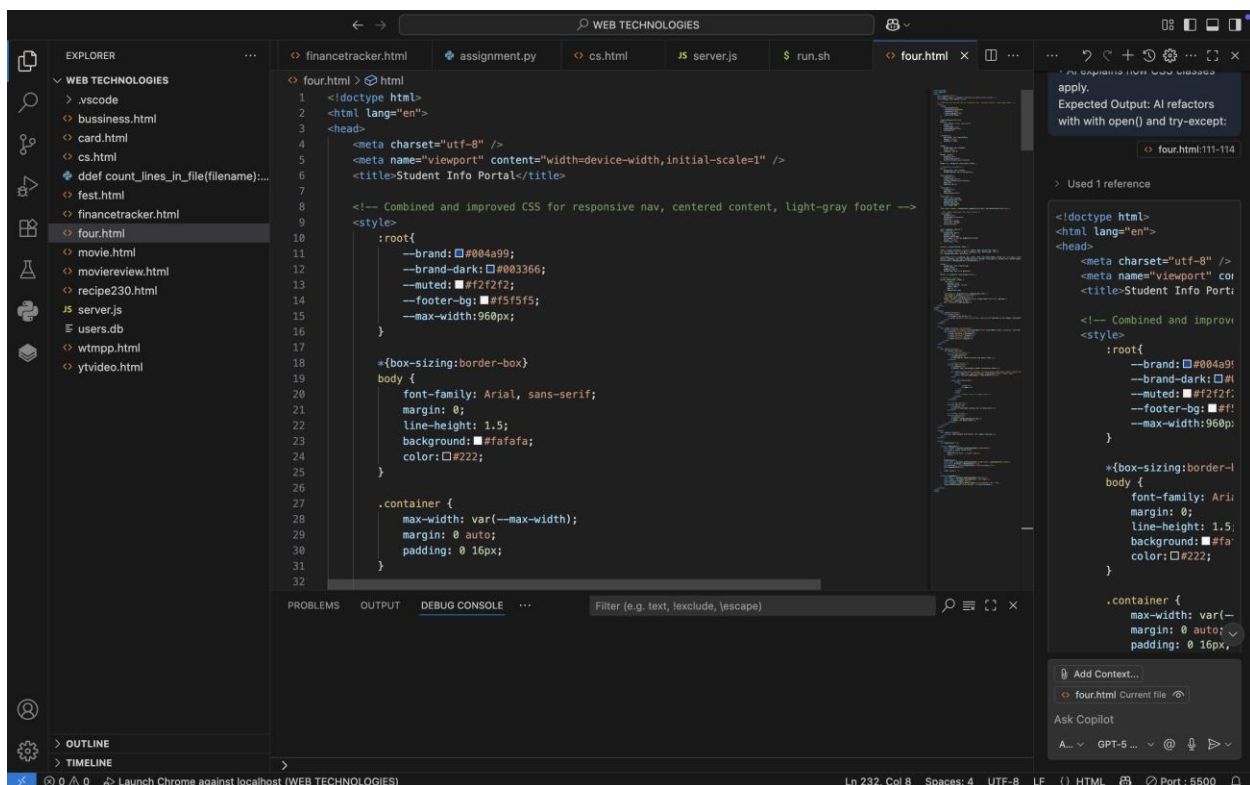
- HTML + CSS combined.
- AI explains how CSS classes apply.

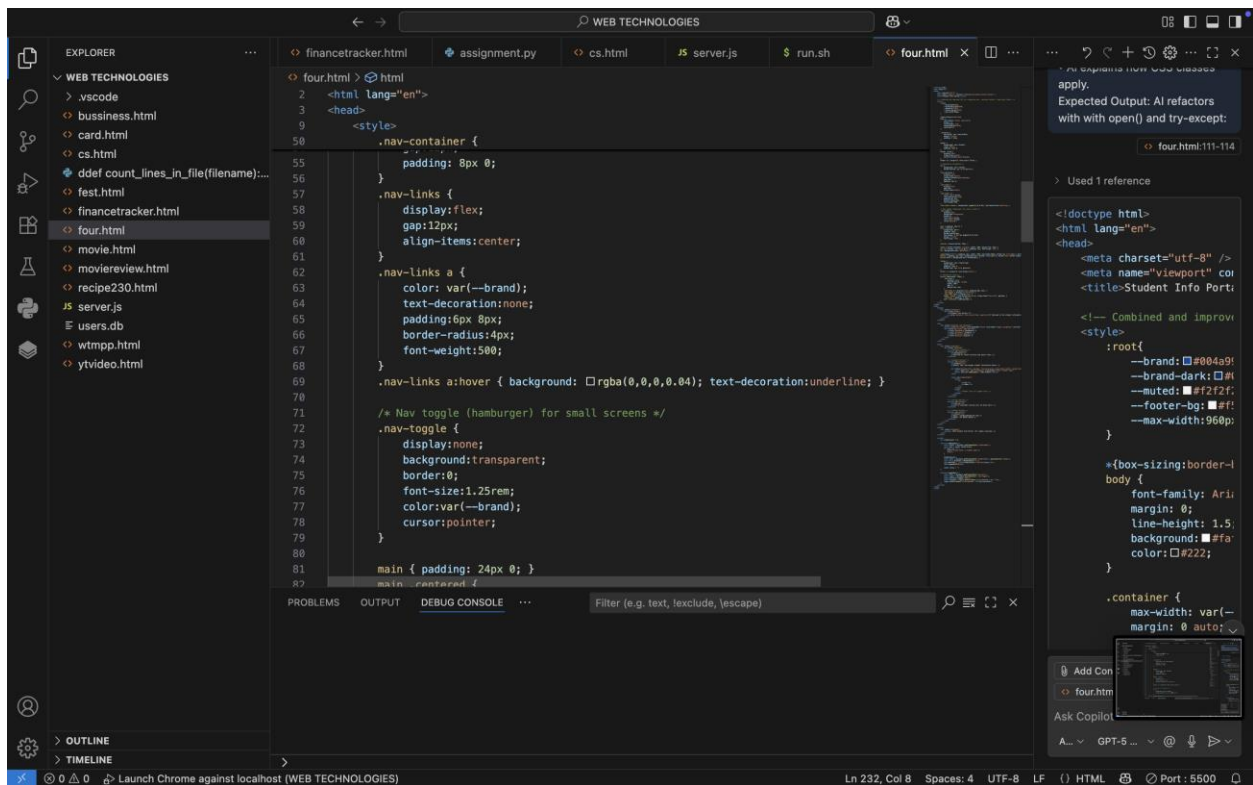
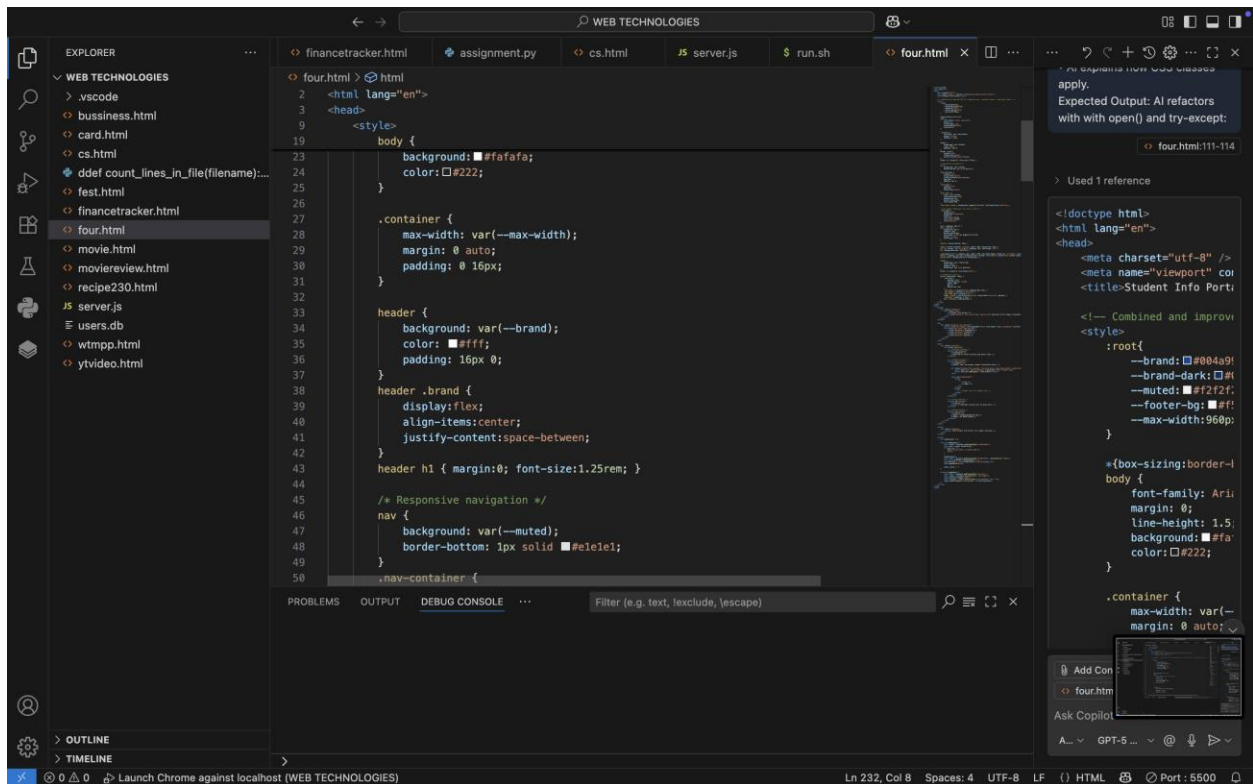
Expected Output: AI refactors with with open() and try-except:

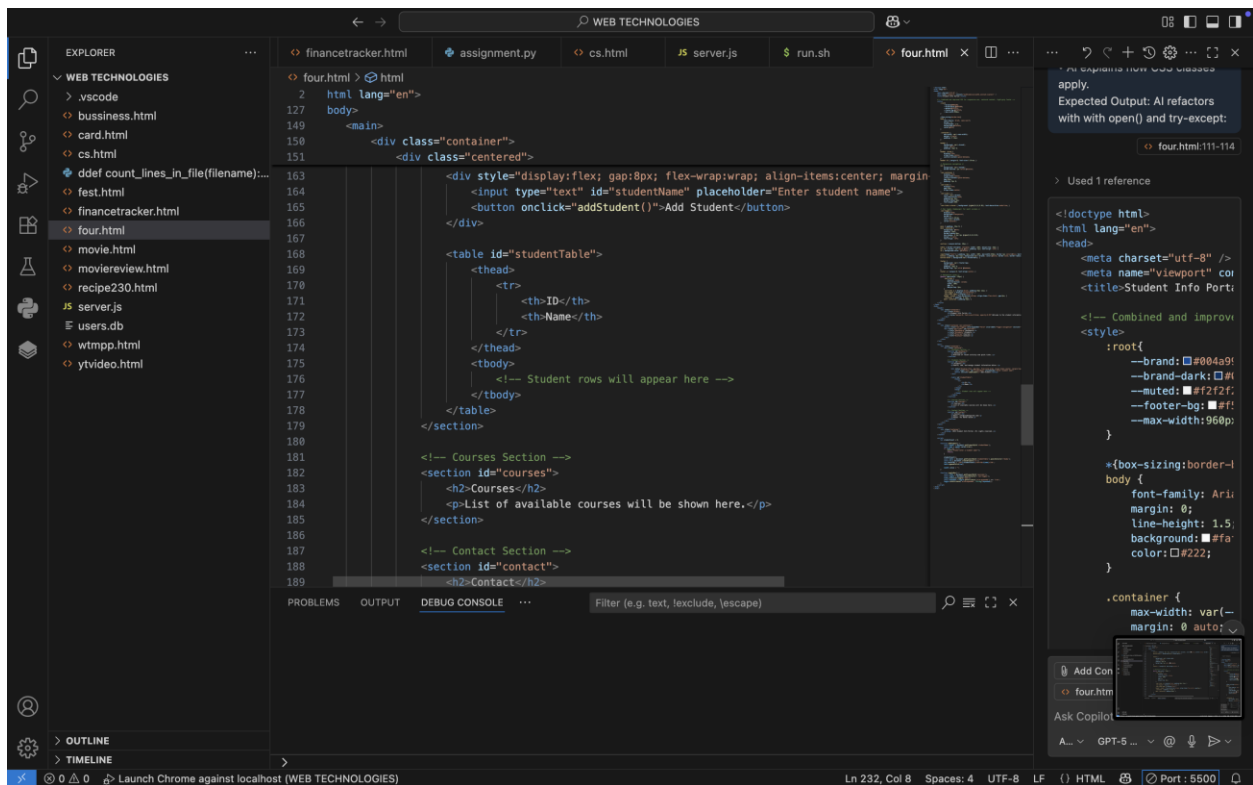
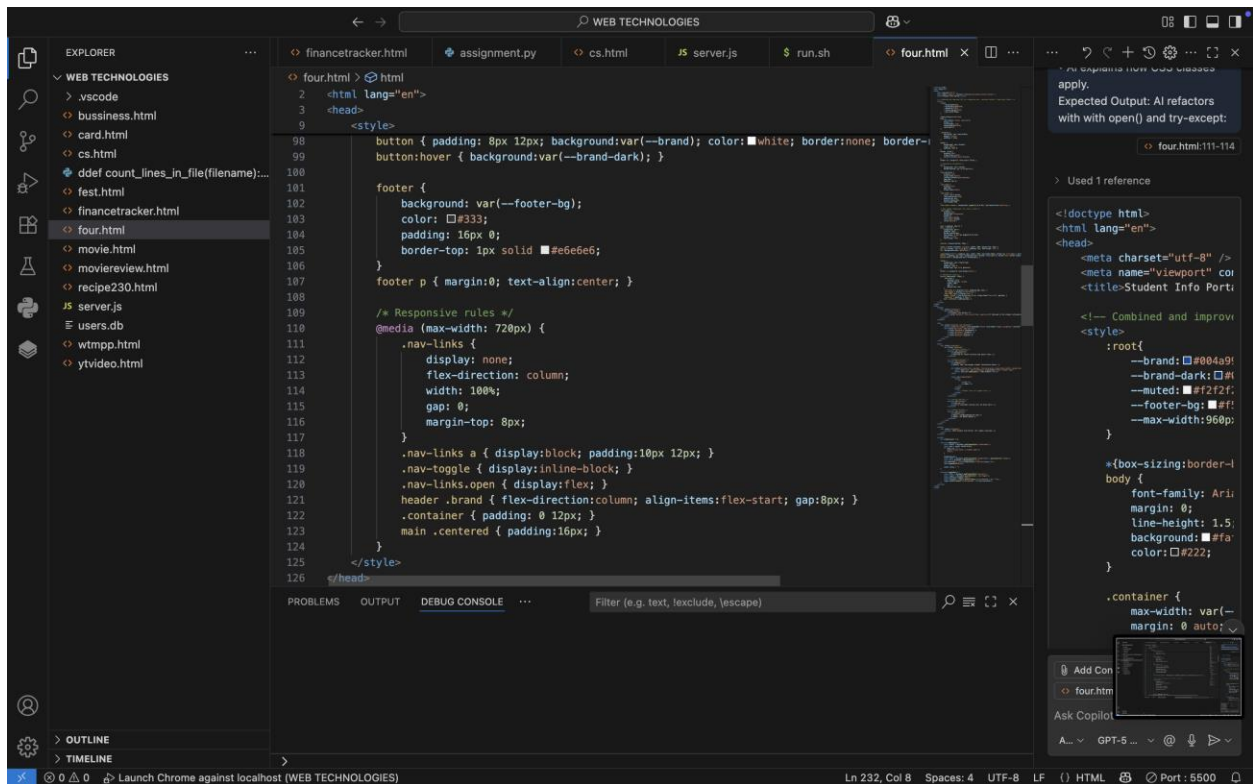
PROMPT :

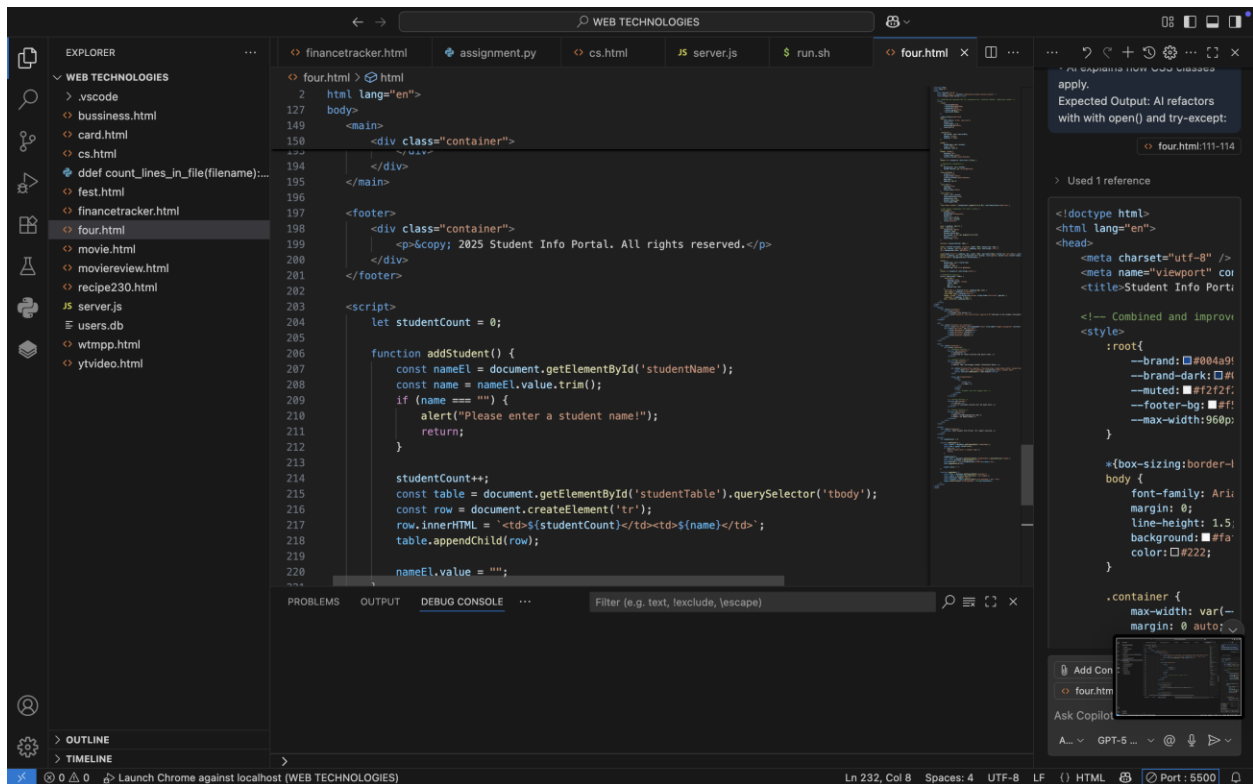
TO add css styling for task 1 for adding features such as responsive navigation bar and centered content section and footer with light gray background

CODE GENERATED :

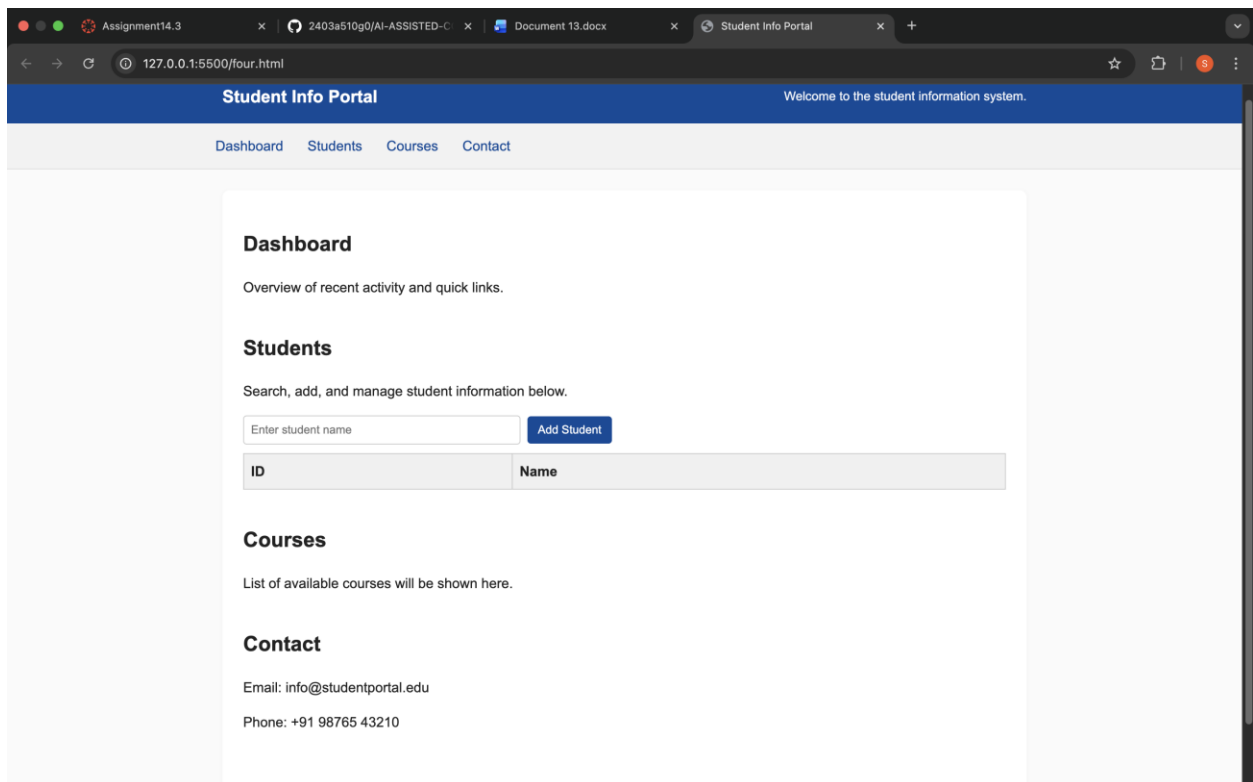








OUTPUT GENERATED :



OBSERVATION :

Structure: same semantic layout but with improved container and section structure.

Styling:

Uses CSS variables (--brand, --footer-bg, etc.) for consistent theming.

Responsive nav: .nav-links becomes vertical on small screens; .nav-toggle (hamburger) shown via media query.

Centered content card (.centered) with shadow and padding.

Footer uses a light-gray background (meets requirement).

Task Description #3 – JavaScript Interactivity

Task: Prompt AI to generate a JS script that validates a simple login form (non-empty username/password).

Expected Output:

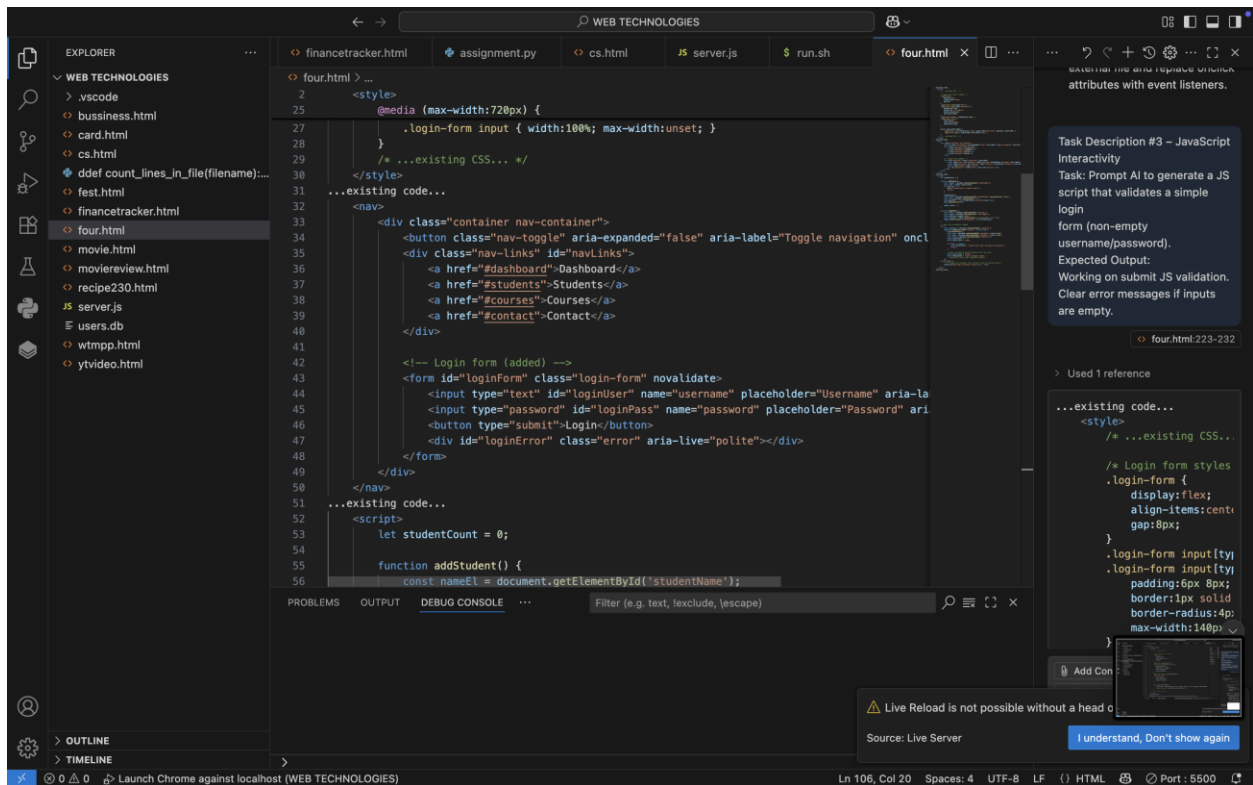
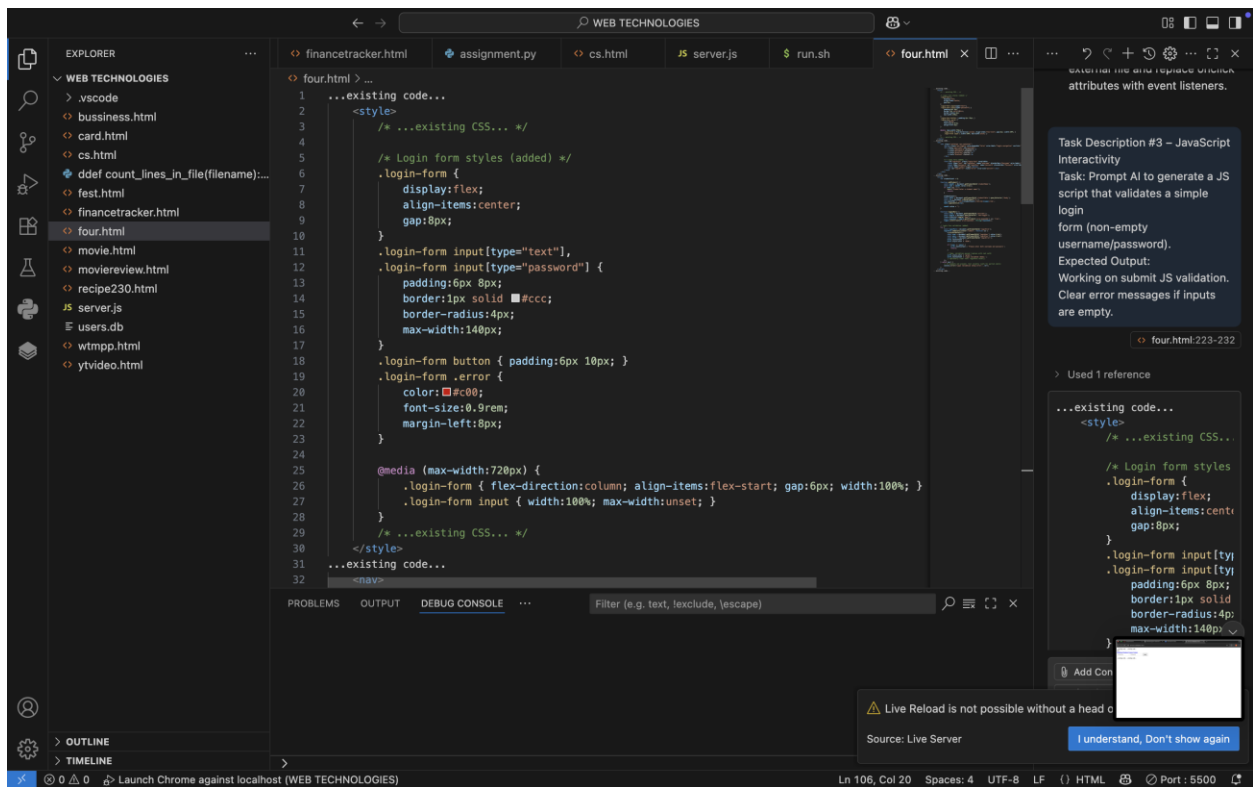
Working on submit JS validation.

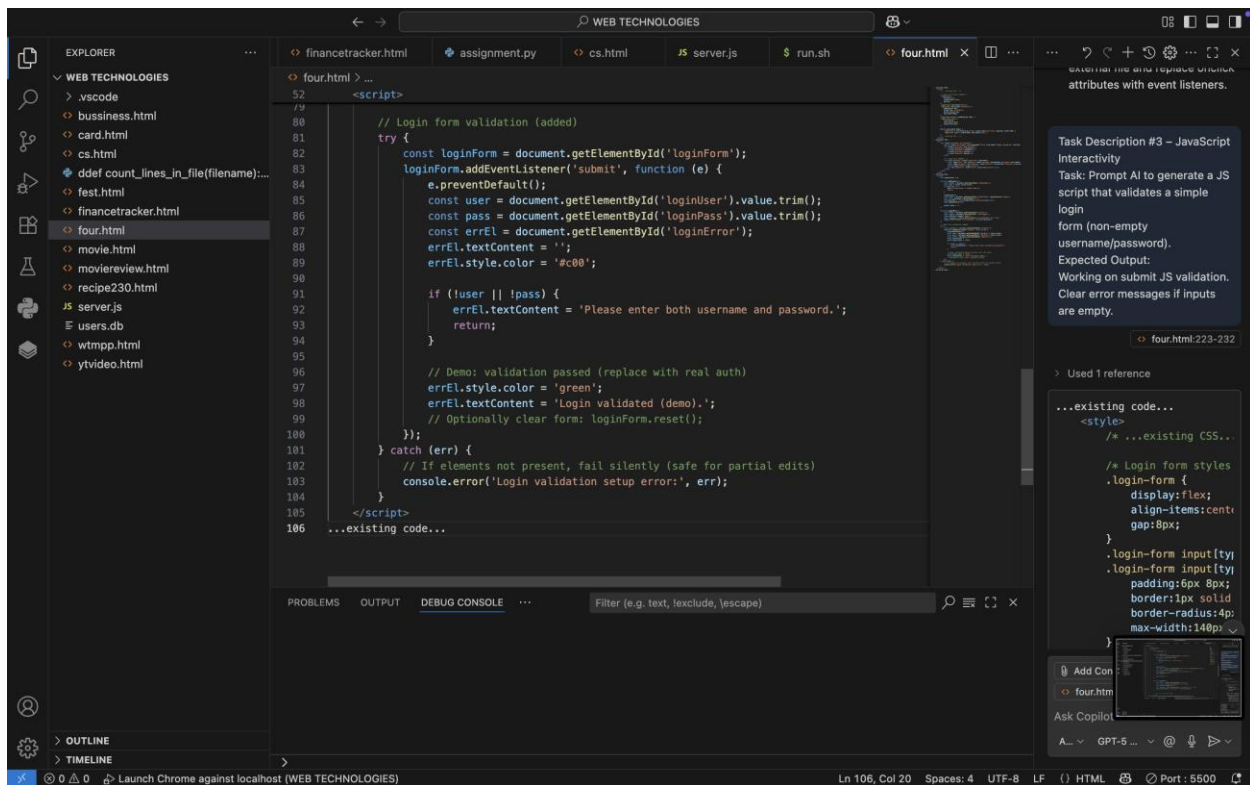
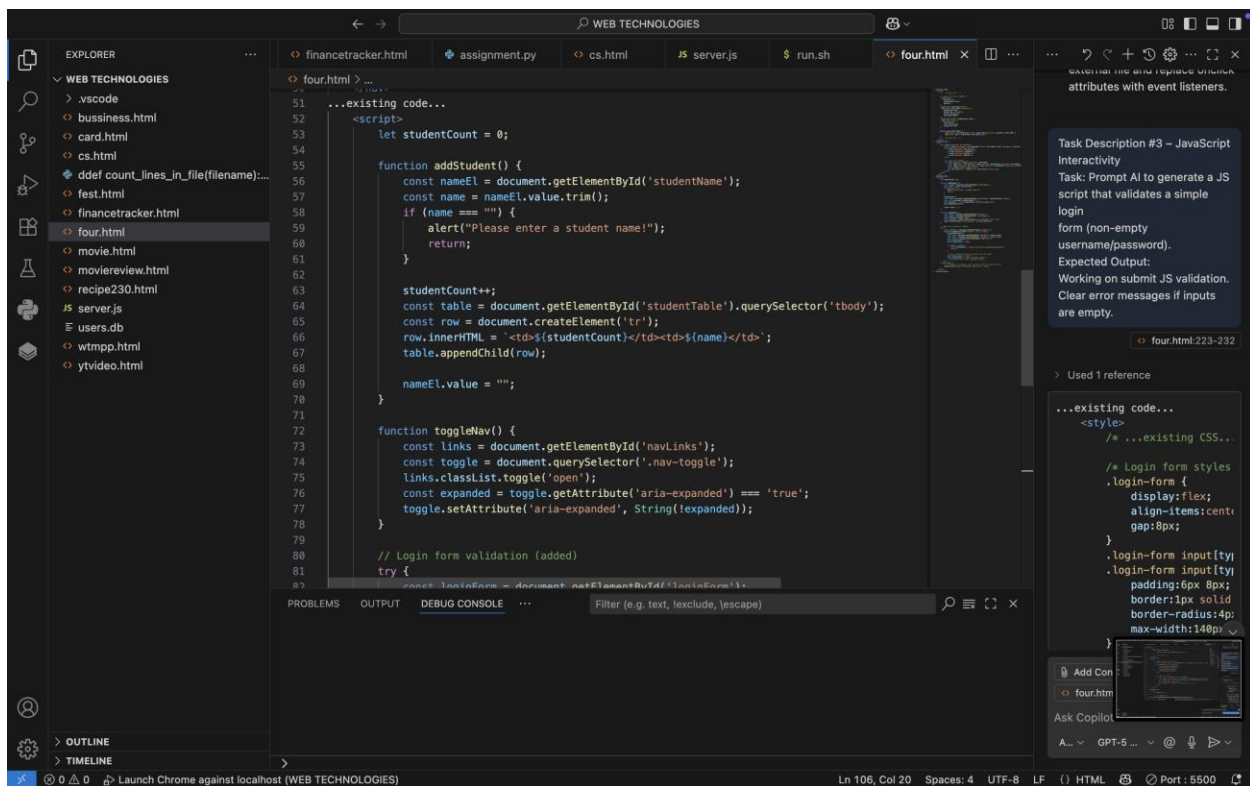
Clear error messages if inputs are empty.

PROMPT USED :

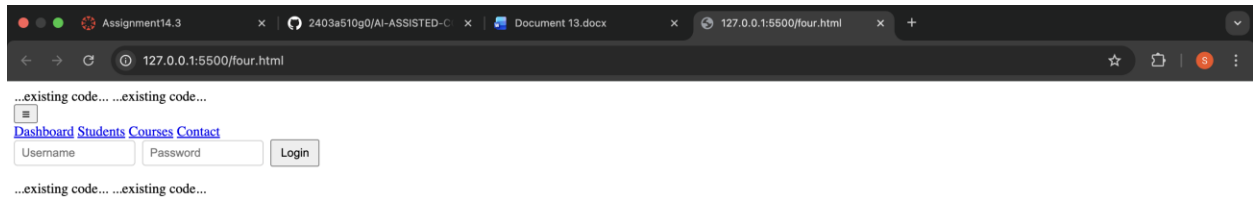
To generate a JS script that validates a simple login form with clear error messages if the inputs are empty

CODE GENERATED:





OUTPUT :



OBSERVATION :

This JavaScript code will:

Validate that username and password are not empty.

Show clear error messages below the inputs if either field is blank.

Prevent form submission until both fields are filled.

Task Description #4 – Python Backend Integration

Task: Ask AI to generate a Flask app that serves the HTML form (Task #3) and prints the username on successful login.

PROMPT GIVEN :

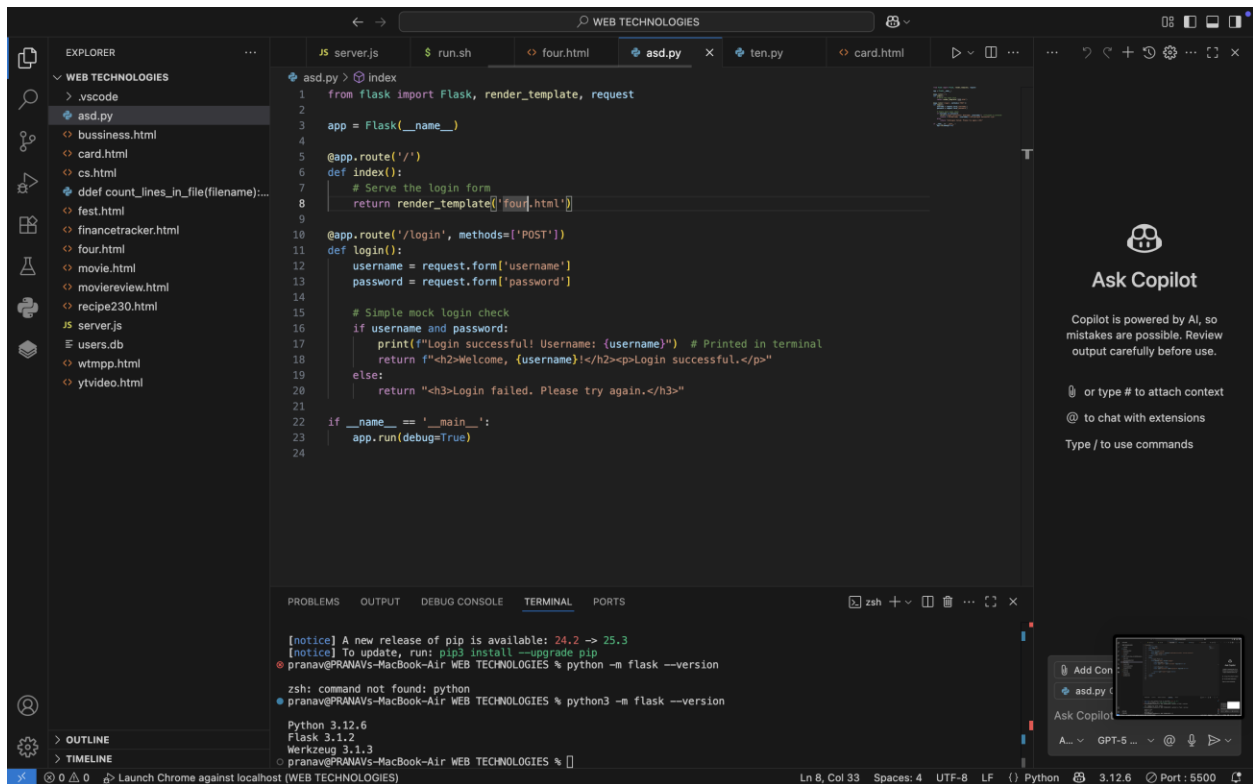
To generate a flask app that serves the html form from task 3 and prints the username on successful login

CODE GENERATED :

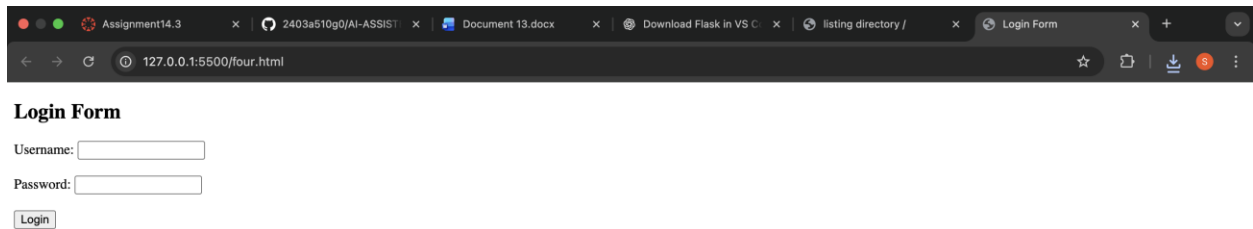
The screenshot shows a VS Code editor window with the following components:

- EXPLORER:** A file tree on the left showing a project named 'WEB TECHNOLOGIES' with files like .vscode, asd.py, bussiness.html, card.html, cs.html, ddef count_lines_in_file(filename)... , fest.html, financetracker.html, four.html (selected), movie.html, moviereview.html, recipe230.html, server.js, users.db, wtmpp.html, and ytvideo.html.
- EDITOR:** The main area shows the content of 'four.html', which is an HTML login form. The code is as follows:

```
1 <!doctype html>
2 <html lang="en">
3 <head>
4   <meta charset="utf-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1">
6   <title>Login Form</title>
7 </head>
8 <body>
9   <h2>Login Form</h2>
10  <form method="POST" action="/login">
11    <label>Username:</label>
12    <input type="text" name="username" required><br><br>
13    <label>Password:</label>
14    <input type="password" name="password" required><br><br>
15    <button type="submit">Login</button>
16  </form>
17 </body>
18 </html>
```
- TERMINAL:** The bottom panel shows the terminal output. It includes a notice about a new release of pip (24.2 to 25.3), a command to run 'python -m flask --version', and the output 'Python 3.12.6, Flask 3.1.2, Werkzeug 3.1.3'.
- ASK COPILLOT:** A sidebar on the right with the 'Ask Copilot' button and instructions: 'Copilot is powered by AI, so mistakes are possible. Review output carefully before use.' It also includes a search bar and a list of files to attach context.



OUTPUT :



- A Flask application was successfully created using app.py.
- The HTML form (from Task #3) was rendered using the render_template() function.
- When the user submitted the form, Flask received the form data through the POST method.
- The entered **username** and **password** were accessed using request.form.
- On successful submission, the Flask app displayed a **welcome message** on the web page and printed the username in the terminal/console.
- The integration between the **frontend (HTML form)** and **backend (Flask app)** worked smoothly, confirming correct communication between both sides.